

Gerenciamento de mudanças e versões

Você vai compreender os fundamentos do controle de versão e do controle de mudanças na gerência de configuração, explorando boas práticas e o uso das ferramentas Subversion, Git, Redmine e Gitlab.

Rodrigo Augusto dos Santos

1. Itens iniciais

Propósito

O entendimento do Controle de Versão e do Controle de Mudanças é essencial para profissionais de Tecnologia da Informação, pois garante o gerenciamento eficiente de alterações em sistemas e softwares, promovendo organização, rastreabilidade e qualidade nos processos de desenvolvimento e manutenção.

Objetivos

- *Distinguir os fundamentos, tipos e boas práticas do Controle de Versão, reconhecendo o uso das ferramentas* Subversion e Git.

- *Analisar os fundamentos, tipos e boas práticas do Controle de Mudança, explorando as funcionalidades das ferramentas Redmine e Gitlab.*

Introdução

Neste conteúdo, será dado enfoque a dois tópicos fundamentais da gerência de configuração: o controle de versão e o controle de mudanças. Esses conceitos são indispensáveis para garantir a integridade, a rastreabilidade e a organização de projetos na área de Tecnologia da Informação, especialmente em ambientes colaborativos e de desenvolvimento contínuo.

O controle de versão será abordado inicialmente, com a apresentação de seus fundamentos, a importância da adoção de versões básicas e funcionais, além de boas práticas que contribuem para um gerenciamento mais eficiente. Esse conhecimento permite ao profissional registrar, recuperar e comparar diferentes versões de arquivos, facilitando o trabalho em equipe e reduzindo riscos de perdas ou inconsistências.

Na sequência, será explorado o Controle de mudanças, que complementa o Controle de Versão ao tratar da gestão estruturada de modificações em projetos. Serão explicados os principais tipos de mudanças e como organizá-las de maneira sistemática, utilizando diretrizes que assegurem a qualidade e a coerência do produto final.

Ao longo do conteúdo, será oferecida uma visão geral sobre quatro ferramentas consagradas no mercado: Subversion e Git, amplamente utilizadas para Controle de Versão; e Redmine e Gitlab, que além de funcionalidades de versionamento, também possibilitam o gerenciamento eficiente de mudanças.

Compreender esses processos e saber utilizar ferramentas adequadas é essencial para qualquer profissional que atue com desenvolvimento de sistemas, testes, manutenção ou qualquer atividade que envolva a evolução controlada de produtos de software. Este conteúdo proporcionará uma base sólida para aplicar esses conceitos na prática profissional.

O controle de versão e seus fundamentos

Vimos anteriormente que várias coisas se classificam como itens de configuração, e que nosso foco se dará sobre aquelas compostas por software (código-fonte, módulos, sistemas inteiros etc.) e documentação em geral. Dado o contexto de uso nas organizações em que estes itens de configuração se encontram inseridos, além da relativa facilidade com que podem ser alterados, é natural esperar que ocorram muitas mudanças em suas estruturas, conteúdos e informações de configuração ao longo dos seus ciclos-de vida.

O controle de versão basicamente se focará em um uma coisa: registrar e rastrear as mudanças que ocorrem nestes itens de configuração. Estas são capacidades fundamentais em um projeto de software, ou mesmo em empreendimentos considerados mais operacionais, tais como manutenção contínua de software, pois traz uma série de benefícios, entre os quais:

Múltiplos membros trabalhando simultaneamente em um mesmo projeto

O controle de versão é fundamental para a colaboração ordeira e organizada entre membros de um mesmo projeto ou empreendimento operacional, pois permite que cada membro edite sua própria cópia de itens de configuração, compartilhando estas cópias com o restante do time quando necessário ou quando certo de que as cópias são estáveis o bastante.

Permite a integração do trabalho que foi realizado simultaneamente, mas separadamente

O controle de versão permite a junção do trabalho realizado por membros diferentes em uma única versão unificada. Permite ainda a visibilidade sobre conflitos de edição, ou seja, os casos em que membros diferentes fazem alterações diferentes em um mesmo trecho de código-fonte, por exemplo. Isto gera subsídio para a tomada de ação no sentido de eliminar o conflito. Embora este ponto seja mais fácil de ser atingido no controle de versão distribuído, também é possível de ser atingido no controle de versão centralizado.

Permite manter o histórico dos itens de configuração

Provavelmente o benefício mais tangível do controle de versão, é uma espécie de seguro contra alterações problemáticas, corrupção não esperada do conteúdo de um item de configuração, perda de dados, etc. O controle de versão permite o levantamento do histórico dos itens de configuração, além de possibilitar a remediação de alterações, reabilitando uma versão que se sabe ser estável. Também importante é a capacidade de saber quem, quando e onde realizou a alteração que causou problemas, permitindo a tomada de ação corretiva e preventiva.

Tais capacidades normalmente serão fornecidas por algum sistema de controle de versão, que por sua vez fornecerá dois elementos chave: o front-end, ou seja a interface com a qual os usuários irão interagir diretamente; e um back-end, representado por alguma forma banco de dados contendo todas as edições e versões históricas dos itens de configuração em questão. A estes bancos de dados dá-se o nome de **“repositório”**.

Temos outros conceitos fundamentais habilitadores dos sistemas de controle de versão. Veja a seguir.

Conceitos habilitadores dos sistemas de controle de versão

- **Cópia de trabalho (working copy, sandbox ou checkout):** itens de configuração que foram selecionados por um usuário e que agora se encontram bloqueados para edição para aquele usuário, enquanto são realizadas as alterações necessárias. Estas são as cópias locais em que o usuário estará trabalhando.

- **Confirmação (Commit):** também conhecido como "realizar um commit", é uma ação de confirmação, sendo uma ação deliberada por parte de um usuário para as mudanças em um item de configuração que ele tenha realizado, enviando-as ao servidor, tornando-as oficiais e disponíveis a outros usuários.

- **Grupo de mudanças (changeset):** é um novo registro gerado automaticamente no repositório para cada commit realizado. Cada changeset ou grupo de mudanças pode conter inúmeras modificações em um ou mais itens de configuração, e seu tamanho e extensão depende de quão frequentemente os usuários escolhem realizar commits. Por exemplo, um usuário pode escolher realizar commits para cada alteração significativa, o que implica em vários commits em um curto espaço de tempo, como minutos ou horas, ou pode escolher realizar commits para períodos maiores de tempo, incluindo dias ou até semanas.

- **Revisão (revision):** Número identificador único dado a cada changeset. A idéia dos revisions é facilitar a rastreabilidade das mudanças.

- **Etiqueta (tag ou label):** Uma revisão que foi congelada, normalmente se tornando "somente-leitura" e que muito provavelmente é candidata a ser liberada para produção.

- **Atualização (Update):** também conhecido como "realizar uma atualização", ocorre quando usuários deliberadamente realizam o download de um changeset do repositório, de forma a possuírem a última versão do código fonte ou documentação.

- **Diferenciação (diffing):** é um mecanismo de visualização das diferenças entre revisões. Facilitando, portanto, a comparação entre versões de um mesmo item de configuração, bem como proporciona conhecimento sobre quem e quando realizou a alteração em questão. A comparação pode ser feita linha a linha para as versões dos itens de configuração, para revisões sequenciais e até mesmo entre duas revisões em qualquer ponto no histórico dos itens de configuração.

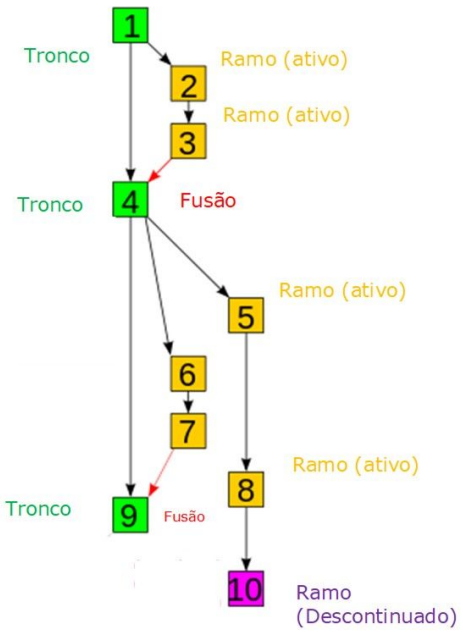
- **Ramificação (branching):** serve para criação de cópias alternativas ou ramos (branches) do código fonte / documentação a ser alterados, permitindo a execução das alterações sem modificação direta do tronco ou código / documentação mestre (master). Branching é importante pois dá subsídio à realização de testes e da consequente estabilização das versões, levando à redução dos riscos resultantes de alterações não-intencionais ou mesmo de alterações planejadas.

- **Fusão (merging):** serve para unificar ou fundir trechos de alterações ou mesmo alterações completas de autoria de usuários diferentes, em uma única nova versão, a ser combinada com a versão mestre disponível no repositório. Como se pode imaginar, é também o mecanismo usado para reunificar o código de um ou mais ramos ao código mestre, uma vez que exista confiança que estes ramos são estáveis e que não irão causar problemas.

- **Conflitos de versão:** ocorrem quando dois ou mais usuários alteram o mesmo trecho ou linha do mesmo pedaço de código-fonte ou documentação, seja por engano, seja quando criam ramos diferentes para isso. Normalmente requerem fusão para eliminação do conflito, no entanto é possível que a ferramenta de controle de versão não consiga sozinha determinar a versão válida, requerendo portanto escolha e intervenção manual.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Assista ao vídeo e aprenda sobre documentação e software.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Diferentes tipos de controle de versão

Repositórios são organizados em pastas (ou diretórios) e arquivos. Conforme usuários vão realizando mudanças nas estruturas destas pastas e / ou arquivos (tais como adição e movimentação de arquivos, deleção de uma pasta, alteração no nome de um arquivo ou qualquer outra coisa que afete o estado daquele objeto) é tarefa do controle de versão manter a rastreabilidade dessas mudanças.

Com o intuito de cumprir este objetivo, os Sistemas de Controle de Versão se organizam de duas maneiras distintas.

Controle de versão centralizado

Passo 1

O sistema de controle de versão irá esperar que o usuário autor das mudanças as envie as mudanças realizadas para “registro oficial”. Em outras palavras, o sistema de controle de versão requer o envio deliberado do changeset pelo usuário-autor, via ações de *confirmação* ou *commit.*

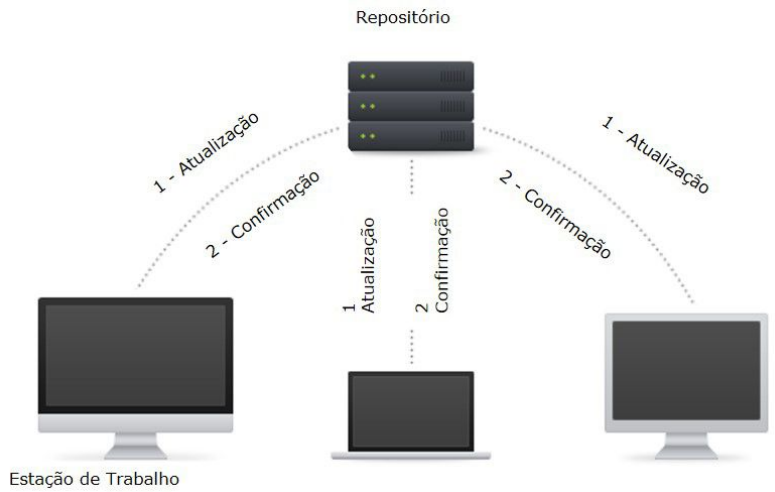
Passo 2

Ocorre quando demais usuários precisam realizar uma atualização ou update, ou seja, precisam deliberadamente realizar o download do changeset, de forma a possuírem a última versão do código fonte ou documentação.

A centralização do controle de versão tem a vantagem de que torna relativamente fácil saber quais partes do código-fonte e documentação encontram-se *bloqueadas para edição* (**check-out** ou **checked-out**) pelos usuários. Estas são as cópias locais que estarão editando. A ferramenta Subversion é um exemplo bem conhecido de implementação do paradigma do controle de versão centralizado.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Controle de Versão Centralizado.

A ordem das ações no controle de versão centralizado é:

1. Atualização (Update)
2. Confirmação (Commit)

A centralização do controle de versão tem a vantagem de que torna relativamente fácil saber quais partes do código-fonte e documentação encontram-se *bloqueadas para edição* (**check-out** ou **checked-out**) pelos usuários. Estas são as cópias locais que estarão editando. A ferramenta *Subversion* é um exemplo bem conhecido de implementação do paradigma do controle de versão centralizado.

Controle de versão distribuído

Difere da abordagem centralizada já que nesta modalidade, além de um repositório centralizado, cada usuário possuirá sua própria cópia local deste repositório inteiro, formando assim uma espécie de rede de repositórios

interconectados. A realização de *commits* não irá enviar as alterações a outros repositórios e torná-las disponíveis aos demais usuários, pois aqui o *commit* se limita a confirmar no repositório local as alterações realizadas pelo autor nos itens de configuração que estavam *checked-out*.

Para que os demais repositórios sejam sincronizados, além das já conhecidas ações de confirmação e atualização, são necessárias ainda mais duas ações adicionais, totalizando quatro ações:

- **Ação *empurra* (push):** o autor das alterações, após commit em seu repositório local, empurra as alterações deste repositório local para o repositório centralizado, disponibilizando-a para os demais usuários.

- **Ação *puxa* (pull):** os usuários puxam ou realizam o download das atualizações agora disponíveis no repositório central para seus próprios repositórios locais. Somente aí a ação de atualização será executada, fazendo com que os seus checkouts, ou as cópias dos itens de configuração em que estão efetivamente trabalhando reflitam as últimas alterações disponíveis.

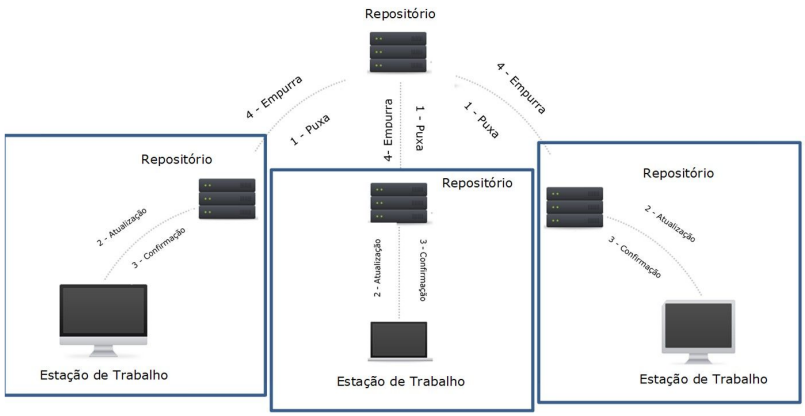
Fica claro portanto que no controle de versão distribuído, as operações de confirmação e atualização apenas movem mudanças entre os checkouts e o repositório local, sem afetar nenhum outro repositório. Já as operações do tipo puxa e empurra movem mudanças entre o repositório local e o repositório central, sem afetar os checkouts.

Da mesma forma que no controle centralizado de versão, no controle de versão distribuído também existe potencial para conflitos de versão, ou seja, situações em que dois ou mais usuários realizem alterações em uma mesma linha de código ou documentação antes de.

A ferramenta Git é um bom exemplo de implementação do paradigma distribuído.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Controle de versão distribuído.

A ordem das ações no controle de versão distribuído é:

1. Puxa (Pull)
2. Atualização (Update)
3. Confirmação (Commit)
4. Empurra (Push)

Boas práticas no controle de versão

Dada o emprego muito específico do Controle de Versão, boas práticas existem e são recomendadas que sejam adotadas. Cobriremos aqui algumas práticas gerais, bem como práticas específicas, tanto para o Controle de Versão Centralizado quanto para o Controle de Versão Distribuído. Fazem parte destas práticas:

Usar mensagens de confirmação que sejam o mais descritivas possível

A cada commit, é importante que seja redigida uma boa mensagem que encapsule bem o significado daquela confirmação. Isto não só é útil ao próprio autor da mudança, já que conseguirá mais facilmente recordar e rastrear a evolução do seu próprio trabalho, mas também irá ser útil a outros usuários que estão a examinar a mudança, já que a descrição indica o propósito da mudança.

Trate cada confirmação de forma atômica

É interessante que cada commit tenha um propósito claro e que o implemente de forma completa, sem dependências, construindo portanto **uma unidade lógica**. Isto facilita a localização de mudanças relacionadas com alguma funcionalidade específica ou correção de defeito, pois as disponibiliza em um local único. A utilidade do controle de versão é diluída se um commit atende a múltiplos propósitos ou se código ou documentação para um propósito em particular se encontram espalhados ao redor de vários commits. Obviamente pode ser impraticável em cem porcento do tempo realizar commits atômicos, mas na medida do possível é importante procurar fazê-lo, pois esta organização renderá dividendos no futuro.

Evite realizar confirmações de maneira indiscriminada

É interessante que a cada commit, arquivos específicos sejam fornecidos para o commit, evitando-se realizar commits gerais, que irão confirmar todo e qualquer arquivo que tenha sido alterado. Evitar commits completos desnecessários é importante pois evita-se confirmar arquivos que tenham sido alterados de maneira temporária ou que ainda não estejam completos e maduros para o commit.

Incorpore mudanças de terceiros frequentemente

Procure trabalhar com versões de arquivos atualizados o máximo possível. Isto evita situações custosas, como por exemplo aquelas em que alguém trabalhou anteriormente na edição do mesmo trecho do item de configuração em que você está prestes a trabalhar mas que você não atualizou antes de começar tais edições. O resultado disto será muito provavelmente um conflito de versão que exigirá intervenção manual para ser resolvido.

Compartilhe suas próprias mudanças frequentemente

Uma vez que você tenha gerado alterações constituintes de uma unidade lógica, procure compartilhá-las de forma imediata. A razão para isso é basicamente a mesma do ponto anterior, já que alguém poderá trabalhar no mesmo trecho em paralelo, exigindo conciliação posterior do item de configuração.

Coordene com seus pares

Procure estar informado bem como informar seus pares a respeito dos itens de configuração em alteração. Novamente, a ideia é evitar conflitos de versão, logo é interessante que se coordene o início e o fim de alterações em cada item de configuração, ou ao menos o mais importantes.

Evite confirmar arquivos gerados ou não-editáveis

Arquivos como PDFs, e arquivos resultantes de compilação não são destinados à edição por pessoas. Assim, pode-se argumentar que não precisam ser inseridos em um repositório de configuração. Embora isto possa ser um ponto polêmico, é importante reconhecer que os sistemas de controle de versão normalmente dispõem de capacidades para ignorar dados tipos de arquivo. Estas funcionalidades existem por um motivo: evitar situações como conflitos, bancos de dados sobrecarregados com arquivos desnecessários e que reduzem a performance do sistema etc. É interessante tecer considerações a este respeito, de forma a tomar uma decisão consciente e que faça sentido quanto à destinação deste tipo de arquivo.

Invista no conhecimento da funcionalidade de fusão

Conflitos são inevitáveis, e por melhor que seja o controle de versão, eles irão ocorrer em algum momento. Boa parte da resolução de conflitos recairá de alguma forma sobre o mecanismo de fusão (merge) disponível. Hoje em dia, é comum inclusive que o sistemas de controle de versão aceitem ferramentas de fusão de forma “plug and play”, ou seja, você escolhe a que melhor atende suas necessidades. Assim sendo, teste e entenda bem o funcionamento destas ferramentas, de forma a escolher uma que lhe dê confiança e conforto no trabalho.

Habilite notificação por e-mail

Esta é uma forma de automação simples mas poderosa, pois toda vez que algum usuário empurrar ou confirmar alterações, todas as partes interessadas serão notificadas. Isto proporciona ganho de eficiência à medida que a consciência situacional também aumenta.

Identificar e conhecer a ferramenta Subversion

Assista ao vídeo e aprenda sobre os dois tipos de controle de versão.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

A ferramenta Subversion, mais popularmente conhecida simplesmente como SVN é uma ferramenta de controle de versão com longo histórico de uso pela comunidade de Gestão de Configuração, remetendo a meados da década de 2000. Por ser uma ferramenta com amplo histórico, implementa suas funcionalidades

de maneira considerada tradicionalista, ou seja, não necessariamente representa o estado da arte, sendo hoje considerada praticamente como uma ferramenta de backup para diretórios, documentos e outros arquivos.

A estrutura do repositório do Subversion tem no tronco o principal ramo ou linha de desenvolvimento, ou seja, ele é o ponto central ou alvo de onde todos os demais ramos ou linhas paralelas de desenvolvimento (também chamadas de ramos particulares) surgem e eventualmente se fundem novamente. O modelo de ramificação do SVN aliás representa uma das maiores reclamações dos usuários, pois toda vez que uma parte do repositório central é ramificada e checked-out, ela se torna parte do diretório local no computador do usuário que realizou o procedimento de check-out, bem como se transforma em diretórios no servidor.



Atenção

Este modelo tende a causar conflitos conhecidos como conflitos de árvore (tree conflicts). Estes conflitos ocorrem quando mudanças são realizadas nesta estrutura de diretórios, ocorrendo, portanto, frequentemente. Outro ponto que atrai reclamações de usuários atuais é o fato de o SVN ser centralizado, o que se traduz na necessidade de estar conectado ao servidor para realizar operações de commit, o que na prática, faz com que o SVN se torne inútil caso essa conexão seja perdida.

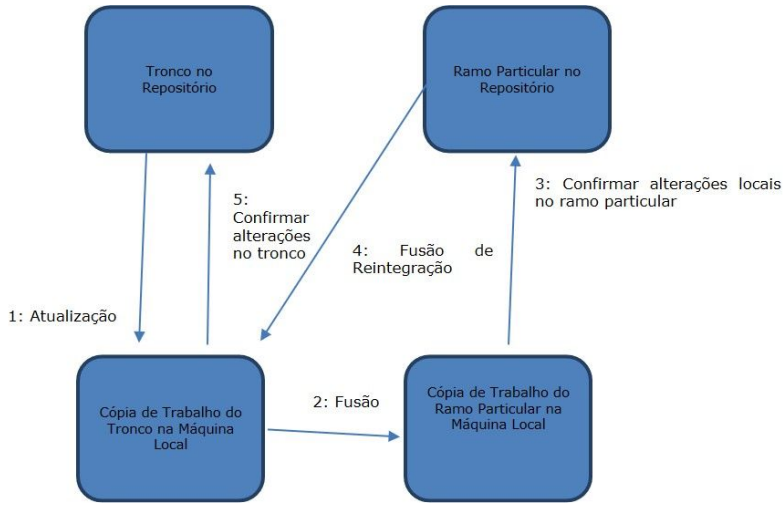
Ainda pelo fato de ser uma ferramenta centralizada, o SVN, além de precisar ser disponibilizado no servidor, precisa também ser disponibilizado nos computadores clientes. O aplicativo SVN mais comum para o papel de cliente é o TortoiseSVN, mas outros existem. O SVN encontra-se disponível para Windows e Linux. No caso do Windows, a instalação é especialmente fácil, pois várias versões que dispõem de interface gráfica existem, tal como a CollabNet. No caso do Linux, a instalação e operação geralmente requerem extenso uso de linha de comando via Terminal.

Do ponto de vista moderno, o uso do SVN ainda se justifica pelo baixo custo, pois a ferramenta em si é open source, sendo que as organizações precisam apenas se preocupar é investir em capacitação, infraestrutura física para hospedagem dos servidores envolvidos. Outro ponto é que muitas organizações que um dia estiveram na vanguarda da adoção da gerência de configuração incorporaram o SVN como ferramenta de controle de versão, sendo que agora, além da ampla base de usuários possuem uma grande quantidade de itens de configuração armazenados pela ferramenta, exigindo grande esforço de migração.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Fluxo operacional da Ferramenta SVN.

Identificar e conhecer a ferramenta Git

Git é uma ferramenta de controle de versão distribuída que pensa nos dados sob seu controle de maneira diferente da maioria das demais ferramentas de mesmo propósito. Enquanto ferramentas como o Subversion por exemplo considera estes dados como um grupo de arquivos e pastas e controla as mudanças feitos em cada um deles ao longo do tempo, o Git na verdade pensa nos dados como fotografias (snapshots) de um sistema de arquivos em miniatura.

Já que o Git é distribuído, existe pouca ou nenhuma latência de rede, já que cópias dos arquivos com todo os seus históricos correspondentes se encontram em repositórios locais nos discos rígidos dos computadores dos usuários. Isto facilita ações, tais como operações de diferenciação entre arquivos, já que não é necessário acessar servidores remotos, fazendo assim com que estas operações pareçam ser instantâneas. Outra vantagem é a possibilidade de trabalhar boa parte do tempo de maneira offline ou desconectada.

O git implementa três estados em que um arquivo pode se encontrar:

Confirmado (committed)
Significa que o dado se encontra armazenado e a salvo no repositório ou banco de dados local.

Modificado
Significa que o arquivo foi alterado por você, mas ainda não foi confirmado em seu banco de dados local e nem está “marcado” para ser confirmado.

Preparado (staged)
Significa que o arquivo foi modificado e que foi marcado ou preparado para ser confirmado na próxima fotografia.

> O Git na verdade implementa três repositórios. O principal é o Repositório Git, que é o repositório central, onde todos os metadados e o banco de dados de objetos estão armazenados. O repositório de trabalho representa um checkout único de uma versão do projeto, e é um clone do repositório Git, além de ser armazenado localmente no disco rígido dos usuários para eles possam fazer alterações e modificações como queiram.

Assim sendo, todas as vezes em que um commit é realizado, basicamente uma foto de como todos os arquivos no momento da fotografia é obtida e então uma referência é atrelada aquela fotografia. Para manter a eficiência, se durante um novo commit apenas alguns arquivos tiverem mudado, então o Git não armazena novamente os arquivos intactos, e ao invés disso apenas estabece um link para os arquivos anteriores idênticos que já se encontravam armazenados.

O Git trata os dados portanto como uma corrente de fotografias, e toda vez que um commit é realizado, na verdade o Git armazena um objeto que contém um ponteiro ou referência para uma fotografia, nome e e-mail do autor, a mensagem de commit, além de ponteiros para o commit ou commits que antecederam o commit atual (os pais). Esta sistemática também se reflete nas ramificações, pois cada ramo passa a ser somente um ponteiro para um commit, tornando toda a operação muito leve. A ação de fusão (merge) também é facilitada, pois ao invés de requerer que o usuário determine manualmente qual é a melhor base a ser fundida, no Git esta escolha é determinada pela própria ferramenta por meio de uma ação denominada “merge commit”.

O índice (index) ou área de preparação (staging area) é normalmente representado por um arquivo que armazena informações sobre o que irá ser “oficializado” no próximo commit, ou seja, representa o que foi alterado e escolhido para ser “subido” do repositório de trabalho para o repositório git. Esta área pode ser omitida, sendo que é possível marcar todo arquivo alterado automaticamente como “staged”, eliminando-se assim a distinção prática e explícita entre os estados modified e committed, fazendo com que todo arquivo alterado seja marcado para ser confirmado.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Fotografias dos dados armazenados pelo Git ao longo do tempo.

O Git pode ser inteiramente usado via linha de comando ou via uma das várias interfaces gráficas compatíveis disponíveis, embora esta segunda opção tenha a desvantagem de que cada interface gráfica normalmente só implemente um grupo reduzido de funcionalidades disponíveis via linha de comando. O Git encontra-se disponível para Linux e Mac (terminal) e para Windows (prompt de comando e powershell).

Assista ao vídeo e conheça as algumas ferramentas para fazer o controle de versão.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Neste conteúdo você aprendeu sobre controle de versão, seus conceitos fundamentais e boas práticas. Embora seja uma atividade de configuração simples, tem uma importância central ao tema de gestão de configuração, pois fornece a estrutura elementar para habilitação da rastreabilidade e segurança a que a gestão de configuração se propõe a oferecer. Vimos ainda uma introdução básica a duas ferramentas muito conhecidas que implementam o controle de versão, no caso as ferramentas Subversion e Github.

Verificando o aprendizado.

Questão 1

Git é uma ferramenta:

A De controle de mudanças.

B De controle de versão.

C De controle de liberação.

D De controle de configuração.

E De controle de construção.

A alternativa B está correta.

Embora o Git possa ser considerado uma ferramenta de configuração, a resposta mais correta e específica é que é uma ferramenta de controle de versão. É no caso um exemplo de ferramenta distribuída.

Questão 2

Sandbox é outro nome dado a:

A Configuração.

B Versão.

C Checkout.

D Ramo.

E Tronco.

A alternativa C está correta.

Sandbox, checkout e cópia de trabalho são descrições referentes à cópia de um item de configuração em que um usuário está trabalhando localmente no momento.

Questão 3

Commits basicamente são:

A O download de um item de configuração alterado por um usuário.

B A fusão de alterações paralelas a um item de configuração.

C O descarte de alterações em um item de configuração.

D A criação de um ramo no repositório.

E A confirmação de alterações em um item de configuração.

A alternativa E está correta.

Commits confirmam em definitivo as alterações realizadas em um item de configuração.

Questão 4

Não é uma boa prática de Controle de Versão:

A Manter o repositório central atualizado.

B Manter o repositório local atualizado.

C Inserir arquivos não-editáveis nos repositórios.

D Fornecer descrições para commits.

E Tratar commits de forma atômica.

A alternativa C está correta.

Deve-se evitar a inserção de itens não editáveis nos repositórios de controle de versão, afinal eles não podem ser alterados, não havendo portanto benefício claro em seu armazenamento na maioria das vezes.

Questão 5

O controle de versão distribuído é realizado com:

A Quatro ações.

B Uma atividade.

C Três ações e uma atividade.

D Duas atividades.

E Quatro atividades.

A alternativa A está correta.

Quatro ações estão envolvidas no controle de versão distribuído: pull (puxada), atualização, confirmação, empurra (push).

O controle de mudança e seus fundamentos

A natureza das organizações atuais é a de lidar com mudanças frequentes. Mudanças de governo, no mercado, nos requisitos, em regulações, leis etc. Esse contexto de mudanças generalizadas e frequentes obviamente se reflete nos negócios das organizações, requerendo assim projetos, operações e empreendimentos flexíveis, adaptáveis, capazes de responder a essas mudanças. Obviamente, tais mudanças chegarão ao nível dos itens de configuração, requerendo assim níveis de controle adequados.

A fim de dispor desses controles, a gerência de configuração dispõe do controle de mudanças, cujo propósito é o de atuar como “porteiro”, ou seja, estar no comando de todos os pedidos de mudança para um produto, bem como de todas as mudanças implementadas. Para todo e qualquer item de configuração, é preciso que seja possível identificar mudanças em relação aos itens que vieram antes dele. Qualquer mudança precisa ser rastreável, sendo possível ligá-la ao item de configuração que recebeu tal mudança.

> O controle de mudanças, portanto, identifica, documenta, aprova ou rejeita e controla mudanças em itens de configuração disponíveis em projetos, operações etc. Em especial, controla as linhas de base. É iniciado por um evento, que nem sempre é formalizado logo em um primeiro momento, podendo ser inicialmente apenas por uma intenção ou desejo por mudança.

É normal que esse evento seja algo negativo, inesperado, errado, durante o uso do item de configuração, mas pode ser também algo relacionado ao desejo por melhorias gerais. Esses eventos, portanto, podem ser coisas como:

Um cálculo errado
Algo percebido durante a revisão de um documento ou relatório.

Erros em código-fonte
Encontrados durante a realização de auditorias, revisão por pares, refatoração ou algum outro esforço semelhante.

Pedidos por melhorias
Originado do uso corriqueiro do software ou documentação, gerando ideias para melhorias de funcionalidades.

Erros ou defeitos
Encontrados durante testes de rotina, testes de integração ou uso comum.

Desejos por expansão

Gerados uma vez que um produto já finalizado esteja em operação, requerendo inclusão de novas funcionalidades inteiras.

Interações com o service desk

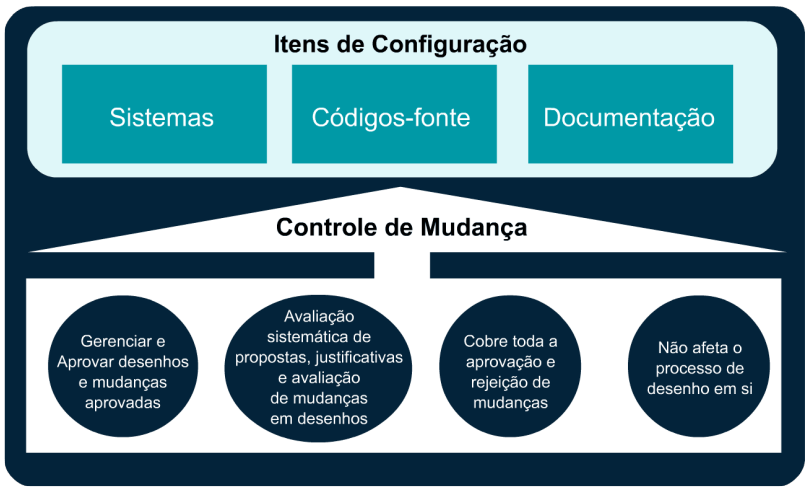
Resultante de problemas no uso do produto ou sistema, geram feedback que pode justificar mudanças.

Mudanças no ambiente técnico

Serviços, produtos ou sistemas normalmente não operam sozinhos, e interagem frequentemente com outros sistemas. Uma mudança em infraestrutura, middleware ou outros componentes relacionados pode desencadear a necessidade por mudanças no item de configuração sob controle de mudanças.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Responsabilidades do controle de configuração

Alguns conceitos fundamentais habilitadores do controle de mudanças são os seguintes:

Mudança

A adição, modificação ou remoção de qualquer coisa nos itens de configuração. Seu escopo pode incluir arquiteturas, processos, ferramentas, métricas, documentações, alterações em serviços de TI e itens de configuração. Em tese, pode ser gerada em qualquer lugar da organização.

Solicitação (ou requisição) de mudança

Mecanismo para formalização de uma proposta de mudança. É, portanto, uma documentação da mudança, e inclui detalhes sobre ela, podendo ser registrada em papel ou eletronicamente. É preciso ainda que seja seguramente armazenado em um banco de dados de algum tipo, de forma a assegurar a persistência da relação entre solicitações de mudança e os itens de configuração em questão.

Linha de base (baseline)

Versão considerada estável, que foi acordada formalmente para um item de configuração. É uma referência no tempo para um item de configuração, descrevendo seu estado naquele momento. Uma nova linha de base só pode ser estabelecida por meio do fluxo adotado para o controle de mudanças.

Comitê de (avaliação de) mudanças

Ocasionalmente representado por uma única pessoa, é mais comum que seja representado por um grupo de pessoas. Suporta a autorização de mudanças e auxilia a gestão de mudanças na avaliação e priorização dessas mudanças. Algumas pessoas que tendem a fazer parte do comitê são gerentes de projeto, representantes do cliente, responsáveis por garantia da qualidade etc. Também conhecido como comitê consultivo de mudança ou comitê de controle de mudanças.

Fica clara aqui a conexão íntima existente entre o controle de mudança e o controle de versão, já que, por dedução, fica claro que, ao se ter mudanças aprovadas e implementas, é inevitável que novas versões de itens de configuração vão também ser geradas.

Aliás, é comum que tanto o controle de mudança e o controle de versão (bem como outras atividades da gerência de configuração) sejam todos implementados e providos por meio de uma única ferramenta ou sistema integrado.

Assista ao vídeo a seguir e compreenda sobre o gerenciamento de mudanças e versões.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Tipos de mudança

Como já deve estar claro, as mudanças têm uma importância e peso muito relevantes durante o ciclo de vida dos itens de configuração, e, por consequência, para a Gestão da Configuração como um todo.

Também deve estar claro que se deve esperar um volume de mudanças considerado alto durante o ciclo de vida de um item de configuração, pois não só estará compreendido o momento de projeto, mas também o tempo de operação destes itens.

Tomando por base essas constatações, pode-se deduzir que é do interesse das organizações a execução de uma tratativa diferenciada de mudanças, em outras palavras, não adotar uma abordagem "*one size fits all*" (um-tamanho-se-adequa-a-todos).

Por mais que o controle de versão purista não preveja explicitamente tipos diferenciados de mudança, frameworks conhecidos no mercado o fazem. Um deles é a *Information Technology Infrastructure Library* (ITIL).

A ITIL, por meio do processo de gestão de mudanças, prevê três tipos de mudança, discriminadas a seguir.

Mudança-padrão

São mudanças do tipo pré-autorizadas, pois normalmente são repetitivas, de baixo-risco e já são bem-testadas, tendo sido executadas inúmeras vezes na organização. Geralmente são empregadas para mudanças originadas de manutenções rotineiras ou corriqueiras em itens de configuração, sistemas, serviços etc, ou seja, situações que são consideradas “bem-entendidas”.

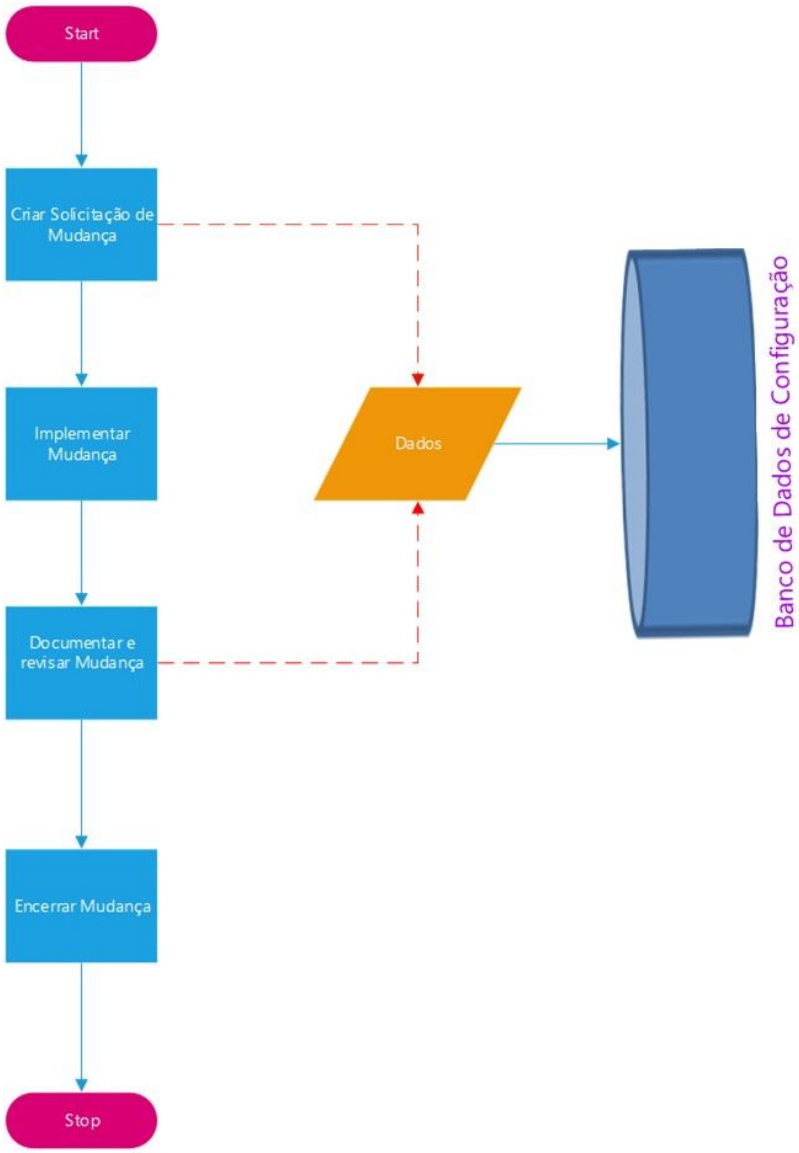
Fica claro que os riscos advindos de mudanças-padrão são baixos, não justificando, portanto, o investimento considerável de recursos financeiros e de tempo representados pelo envolvimento de vários profissionais de diferentes perfis para a avaliação, documentação e implementação dessas mudanças.

Comentário

Tais mudanças normalmente são implementadas por meio de procedimentos conhecidos, repetíveis, muitas vezes roteirizados via algum tipo de checklist.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



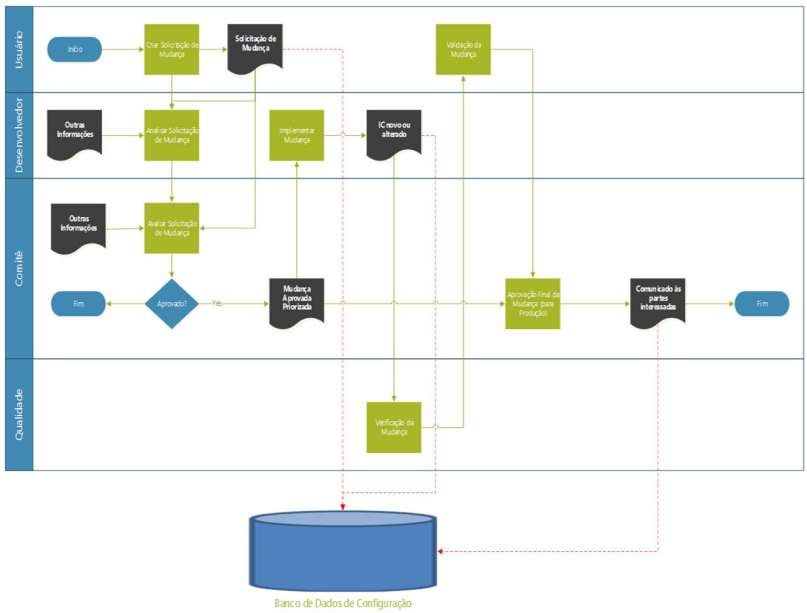
Possível fluxo simplificado para mudanças padrão

Mudança normal

Como o próprio nome diz, é uma mudança comum, que deve receber o tratamento e supervisão esperados para as mudanças. Essa é inclusive a variante de mudança que, caso o controle de mudanças esteja bem definido e maduro, deverá gerar o maior volume de solicitações de mudança, passando por todo o ciclo do controle de mudanças.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Possível fluxo para mudança normal.

Isso implica dizer que serão analisadas e avaliadas de forma criteriosa, em busca de se entender o mérito da mudança, os vários riscos envolvidos, priorizando-a e implementando-a adequadamente, e possivelmente executando-se a verificação (testes de qualidade focados em defeitos) e a validação (normalmente testes dos usuários para assegurar conformidade com requisitos).

Os riscos destas mudanças normalmente são medianos, e o ciclo do controle de mudança ajuda a mitigá-los.

São exemplos de mudanças normais a migração de recursos críticos de informação, migração de sistemas de um ambiente não-operacional para o ambiente operacional, execução de scripts em bancos-de-dados, a liberação de uma nova versão de relatório, entre outros. Mudanças normais geralmente se encontram entre mudanças padrão e mudanças emergenciais em termos de complexidade e riscos.

Mudança emergencial

No ramo da tecnologia da informação, situações inesperadas que causam grande impacto ao negócio tendem a ocorrer com certa frequência.

Exemplo

Falhas em um item de configuração, sistema ou serviço, que como consequência acabam por conduzir à indisponibilidade parcial ou mesmo total de uma ou mais faces dos negócios (operações, por exemplo), gerando prejuízos materiais, financeiros, de credibilidade e outros.

Também precisamos levar em consideração situações em que, para evitar potenciais disrupções como as previamente mencionadas, haja necessidade de uma ação preventiva, como por exemplo, aplicar um patch para uma vulnerabilidade grave conhecida, ou executar a reconfiguração de uma parte ou de toda a rede corporativa diante da ameaça de um iminente ataque malicioso etc. Quando isso ocorre, um tipo especial de mudança é requerido: A **mudança emergencial**.

Normalmente, as mudanças emergenciais irão dispensar qualquer esforço de documentação inicial, indo diretamente para implementação. Dispensarão, muito provavelmente ainda, avaliação pelo Comitê de Mudanças, passando ao invés disso por um processo de aprovação rápido, que normalmente envolve perfis da alta administração, tais como gerentes sênior, diretores, vice-presidentes e outros, garantindo assim a agilidade necessária, ao mesmo tempo em que se mantém um mínimo de supervisão.

Mudança emergencial

As mudanças emergenciais são basicamente o oposto das mudanças padrão: geralmente, mais do que se caracterizarem por trazerem riscos médios a altos, exigem implementação o mais rapidamente possível, de forma que também possam ser operacionalizadas com grande rapidez. São, portanto, mudanças para “situações de crise”, e que requerem procedimentos à altura.

Atenção

Essas aprovações são, portanto, incômodas de serem obtidas, pois acarretam perguntas acerca da mudança, constituindo ainda mecanismo que desencoraje a tratativa de qualquer mudança como sendo emergencial.

Por vezes, uma variação do comitê de mudanças, conhecido como comitê de mudanças emergenciais, pode ser estabelecida para lidar com mudanças emergenciais. Como explanado, outra variação comum é para as categorias das mudanças, podendo ser classificadas de acordo com quem iniciou as mudanças, por exemplo: mudanças (vindas) de usuário, mudanças (vindas) de fornecedores, mudanças de TI etc.

Assista o vídeo e saiba mais sobre gestão de mudanças.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Boas práticas no controle de mudança

Um bom controle de mudança é crítico para uma gerência de configuração eficaz, mas também eficiente, pois ajuda a evitar impactos negativos sobre os projetos e operações, tais como atrasos, erros, retrabalho e outros. Existe obviamente uma série de boas práticas que podem ser adotadas para que se obtenha esse controle de mudanças maduro, entre elas:

- **Usar ativamente os metadados disponíveis**: Quando se lida com mudanças, metadados podem e devem ser usados para propósitos analíticos. Isso pode se dar por meio de relatórios ou por busca direta nos bancos de dados que armazenam os metadados.

Informações disponíveis e que conduzem à rastreabilidade são usadas, por exemplo, para determinar em qual ou quais itens de configurações é necessário implementar as mudanças.

Informações sobre variantes ou ramos de um item de configuração também podem ser usadas para determinar se uma mudança tem impactos em vários locais. Ainda é possível usar metadados para determinar

se algum item de configuração possui solicitações de mudança em aberto, tais como mudanças em processo de implementação ou mesmo aguardando a decisão final sobre se serão ou não implementadas.

- **Realizar análise de impacto**: Ao se analisar e avaliar requisições de mudança, é preciso considerar os impactos de tal mudança, como, por exemplo, os custos associados à implementação e à não-implementação. Embora isso pareça lógico, nem sempre é algo fácil de fazer. Um bom começo é adotar algo como um *checklist*.

Análise de impactos

Identificar

- Todos os requisitos afetados ou em conflito com mudança proposta.
- Consequências caso a mudança proposta não seja implementada.
- Possíveis efeitos adversos e outros riscos ligados à mudança.
- Quanto do valor já investido no produto ou item de configuração, alvos da mudança, será perdido se a mudança for implementada — ou se não for.

Checar se a mudança proposta

- Tem algum efeito em requisitos não-funcionais, tais como resiliência, performance, disponibilidade, etc.
- Pode ser introduzida com as tecnologias conhecidas e recursos que se encontram disponíveis.
- Irá requerer níveis de recursos inaceitáveis para desenvolvimento e testes.
- Irá gerar um preço melhor ou mais alto.
- Irá afetar marketing, produção, serviços ou suporte.

Checar se a mudança proposta acarreta adições, mudanças, ou remoções

---

- Interfaces internas ou externas, relatórios, ou armazenamento de dados.
- Desenhos, Código-fonte, *scripts.*
- Planos e especificações de teste.
- Textos de ajuda, manuais de usuários, material de treinamento.
- Planos de Projeto e planos componentes (qualidade, cronograma, custos, outros).
- Outros sistemas, aplicações, bibliotecas ou mesmo *hardware*.

Ferramentas

Redmine

A ferramenta Redmine é uma ferramenta opensource voltada para a gestão de projetos profissional.

Tem em seu núcleo a proposta de rastreamento de aspectos relacionados aos projetos, permitindo a colaboração de times inteiros com relação à execução e progresso de tarefas, defeitos, capacidades e outros itens, além de proporcionar gráficos de Gantt, permitir a criação de wikis de projetos etc. É ainda agnóstica o bastante para se acoplar a vários sistemas de gerenciamento de banco de dados, tais como MySQL e SQL Server.

A Redmine é capaz de se integrar com vários sistemas de controle de versão por meio de um módulo especial, suportando várias ferramentas conhecidas, tais como o Subversion e o Git. No entanto, é na capacidade de rastreamento de problemas (*issues*) que a ferramenta se destaca, pois tal capacidade pode ser, até certo ponto, utilizada como ferramenta de controle de mudança.



Comentário

Aliás, é comum que usuários se perguntem se a Redmine é mesmo uma ferramenta de Gestão de Projetos, ou se na verdade é uma ferramenta de rastreio de problemas. Como é de se esperar, vários campos que gerarão metadados existem, tais como prioridade, datas, tempo estimado, percentual completo etc.

Os problemas (issues) podem ser categorizados da seguinte forma:

É possível inclusive relacionar problemas a outros problemas existentes. Seja como for, o Redmine disponibiliza mecanismos para **rastrear as mudanças** nos status destes problemas.

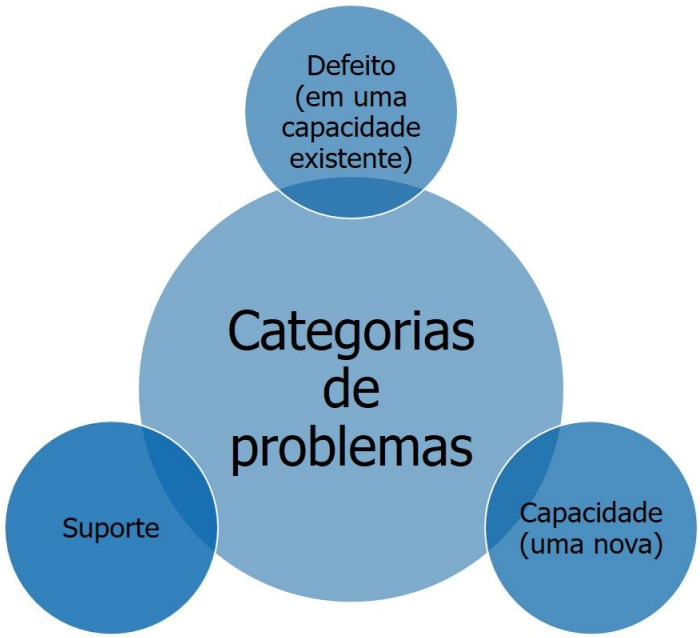
Novas capacidades estão disponíveis e também estão sendo constantemente desenvolvidas e adicionadas ao Redmine via plugins. Se tal capacidade de expansão é vantajosa, por outro lado também pode ser considerada como uma fraqueza do Redmine, pois a ferramenta não é excepcional em nada do que faz, sendo mediana se comparada a ferramentas mais especializadas em um ou alguns poucos aspectos.

Rastrear as mudanças

Esse rastreamento é bidirecional, ou seja, pode informar mudanças a todos os envolvidos quando o usuário altera o problema, ou quando o time de desenvolvimento o atualiza, de acordo com o progresso do trabalho.

Conteúdo interativo

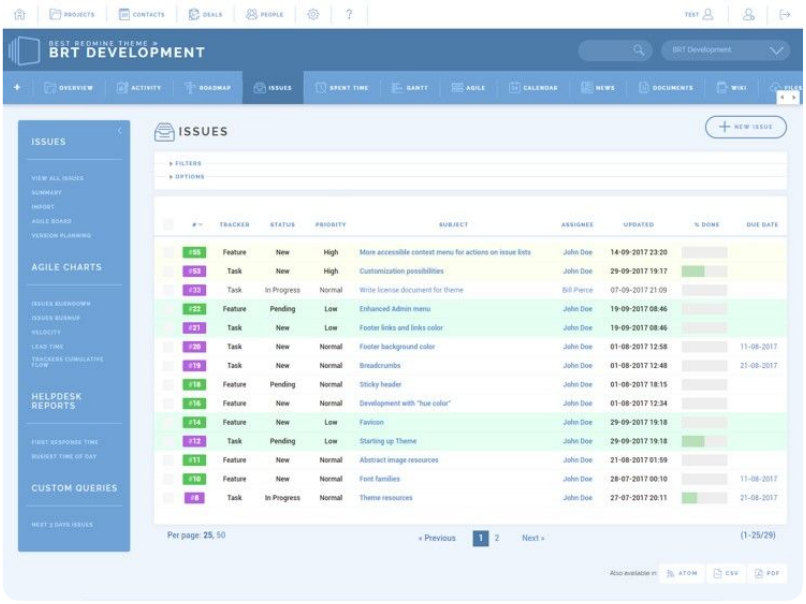
Acesse a versão digital para ver mais detalhes da imagem abaixo.



Outra reclamação constante é a interface desatualizada, além de proporcionar uma experiência um tanto confusa aos usuários.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

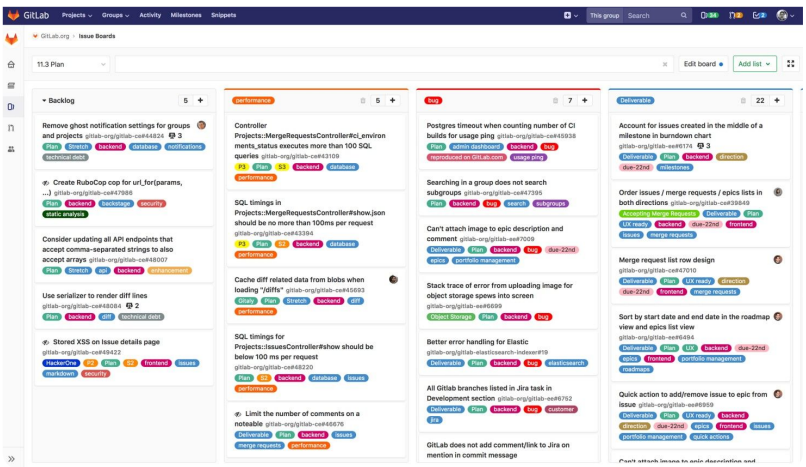


Visão da área de problemas no Redmine.

Gitlab

O Gitlab é primariamente uma ferramenta para controle de repositórios Git. Para isso, é constituído como uma camada construída sobre o Git, e similarmente ao Redmine, suporta desenvolvimento de software como um todo, destacando-se suas capacidades avançadas de controle de versão. O Gitlab pode inclusive ser uma alternativa completa a ferramentas de controle de versão, como o SVN, por exemplo.

> **Conteúdo interativo**
>
> Acesse a versão digital para ver mais detalhes da imagem abaixo.



Fotografias dos dados armazenados pelo Git ao longo do tempo.

Já que suporta o ciclo de desenvolvimento completo, o Gitlab fornece os mecanismos e controles para que, similarmente ao Redmine, exerça o controle de mudança de maneira adaptada.

É interessante notar que, diferentemente do Redmine, existem vários níveis para membros, incluindo um que não custa nenhum valor mensal, até outros que custam até uma centena de dólares mensais.

Similarmente ao Redmine, reclamações com relação à interface e experiência navegacional dos usuários são relativamente comuns, com o adendo de que muitos usuários tendem a reportar bugs nas funcionalidades disponibilizadas.

Outro ponto que merece destaque são os reportes de suporte à comunidade não tão adequados, o que pode ser um fator importante a ser considerado antes da adoção dessa ferramenta.

Assista o vídeo e entenda o que é preciso para a gerência de configuração funcionar bem.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Verificando o aprendizado.

Questão 1

Pode ser usado como ferramenta de controle de mudança:

A Git.

B Gitlab.

C Jenkins.

D SVN.

E CVS.

A alternativa B está correta.

O Gitlab, entre todas as ferramentas listadas, pode ser usado de maneira ao menos parcial para a realização do controle de mudança.

Questão 2

É uma formalização para mudança:

A Comitê de mudança.

B Versão.

C Linha de base.

D Solicitação de mudança.

E Mudança padrão.

A alternativa D está correta.

Uma solicitação de mudança pode ser um documento ou formulário formal físico ou eletrônico.

Questão 3

Que tipo de mudança normalmente implica em pré-aprovação. Estamos falando de:

A Padrão.

B Normal.

C Emergencial.

D Urgente.

E Solicitação.

A alternativa A está correta.

Mudanças-padrão são bem conhecidas e possuem baixo risco; logo, normalmente são pré-aprovadas e não precisam passar por todo o ciclo do controle de mudança.

Questão 4

Não é uma boa prática de controle de mudança:

A Descartar metadados.

B Checar recursos necessários.

C Entender mudanças adicionais.

D Entender a consequência de não implementar.

E Realizar análise de impacto.

A alternativa A está correta.

Os metadados são fonte de conhecimento analítico; logo, podem ajudar a entender o contexto da mudança, incluindo impactos em cascata que ela pode ocasionar.

Questão 5

O comitê de mudanças precisa:

A Sempre ser envolvido em qualquer mudança.

B Ser sempre envolvido em mudanças do tipo emergencial.

C Nunca ser envolvido em mudanças.

D Ser sempre envolvido em mudanças normais.

E Ser sempre envolvido em mudanças padrão.

A alternativa D está correta.

Mudanças normais são as mudanças que passarão pelo controle de mudança completo. Assim sendo, o comitê de mudanças, mesmo quando representado somente por uma pessoa, precisa ser envolvido.

Considerações finais

Neste conteúdo, abordamos dois pilares essenciais da Gerência de Configuração: o Controle de Versão e o Controle de Mudanças. Iniciamos com os fundamentos do Controle de Versão, destacando sua importância para o registro, recuperação e comparação de versões de arquivos ao longo do ciclo de vida de um projeto. Foram apresentadas boas práticas que favorecem a organização do trabalho em equipe, a prevenção de conflitos e a integridade dos dados.

Na sequência, exploramos o Controle de Mudanças, um processo que garante a gestão estruturada de alterações em projetos, contribuindo para a manutenção da qualidade e da consistência dos produtos desenvolvidos. Discutimos os tipos básicos de mudança e práticas recomendadas para sua aplicação no ambiente de trabalho.

Apresentamos também quatro ferramentas amplamente utilizadas no mercado: Subversion e Git, voltadas ao Controle de Versão; e Redmine e Gitlab, que, além de funcionalidades de versionamento, oferecem recursos para o gerenciamento eficaz de mudanças. A familiaridade com essas ferramentas proporciona mais segurança e agilidade na condução de processos de configuração em projetos de software.

Ao finalizar este conteúdo, você deve estar apto a compreender a importância desses dois controles dentro da Gerência de Configuração e a reconhecer como sua aplicação impacta diretamente a eficiência e a qualidade dos projetos. Esse conhecimento é fundamental para profissionais que atuam em desenvolvimento, testes ou manutenção de sistemas e desejam aplicar boas práticas na gestão técnica de seus projetos.

Explore +

- Pesquise na internet sites, vídeos e artigos relacionados ao conteúdo visto.

- Em caso de dúvidas, converse com seu professor online por meio dos recursos disponíveis no ambiente de aprendizagem.

Referências

- AIELLO, B. **Configuration Management Best Practices**. 1 ed. Pearson, 2013.

- BLISS, N. **SVN / Subversion Version Control Crash-Course & Quick Reference**. 1 ed. CreateSpace Independent Publishing Platform, 2013.

- BOURQUE, P.; FAIRLEY, R. **Software Engineering Body of Knowledge (SWEBOK)**. 3 ed. IEEE Computer Society, 2017.

- EVERTSE, J. **Mastering Gitlab 12**: Implement DevOps culture and repository management solutions. 1.ed. PACKT Publishing, 2019.

- GARNER, J. **GIT: The Ultimate Guide for Beginners: Learn Git Version Control**. 1 ed. Independently Published, 2020.

- HAAS, J. **Configuration Management Principles and Practice**. 1.ed. Addison Wesley, 2003.

- LESYUK, A. **Mastering Redmine**. 2.ed. PACKT Publishing, 2016.